

## מבחן במבוא למדעי המחשב – 67101

מועד ב' 22.2.19

### מורי הקורס: אביב זהר וג'ף רוזנשיין

במבחן זה שני חלקים. יש לענות על כל 10 השאלות בחלק א', ולבחור 4 מתוך 6 השאלות בחלק ב' (אין לענות על יותר מ-4 שאלות בחלק ב'). יש לענות על כל השאלות בגוף השאלון – בתוך "קופסאות התשובה" בלבד. אין להגיש דפי טיוטא – הם לא יבדקו.

חומר מותר לשימוש בבחינה: אין. משך הבחינה: 3 שעות.

### חלק א' – שאלות סגורות (40% -- יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב-4 נק')

בחלק זה של המבחן יש לכתוב את התשובה הסופית בלבד בתוך המשבצת המתאימה. אין צורך לנמק.

| שאלה | תשובה |
|------|-------|
| 6    |       |
| 7    |       |
| 8    |       |
| 9    |       |
| 10   |       |

| שאלה | תשובה |
|------|-------|
| 1    |       |
| 2    |       |
| 3    |       |
| 4    |       |
| 5    |       |

### סמנו כאן על אילו שאלות בחרתם לענות בחלק ב' :

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|

(הקיפו בעיגול)

1. מה תדפיס התוכנית הבאה?

```
def foo(x,y,z):
    try:
        x = [0]
        y[0] = 1
        z[0] = 2
        w[0] = 3
    except:
        pass
    return z

x,y,z = [None],[None],[None]
w = z[:]
foo(x,y,z)
print(x,y,z,w)
```

2. מה תדפיס התוכנית הבאה?

```
def check_it(word,alist):
    x = [(True in [c in pattern for c in word]) for pattern in alist]
    print(False not in x)

check_it("aBcD",["aA","bB","cC"])
check_it("aBc",["aA","bB","cC"])
check_it("aBc",["aA","bB","cC","dD"])
```

3. מה סיבוכיות זמן הריצה של הפונקציה הבאה (כפונקציה של  $lst$ )?

```
def foo(lst):
    count = 0
    while len(lst)>1:
        mid = len(lst)//2
        lst = lst[:mid]
        count += len(lst)
    return count
```

4. מה סיבוכיות זמן הריצה של הפונקציה הבאה (כפונקציה של  $n$ )?

```
def f(n):
    if(n<17):
        return 5
    return f(n-1) + sum(list(range(n+4)))
```

5. מה תדפיס התוכנית הבאה?

```
class Message:
    def __init__(self, text, log):
        self.text = text
        self.log = log

    def copy(self):
        return Message(self.text, self.log)

    def add(self, more):
        self.text += more
        self.log.append(more)
        return self

msg1 = Message("", []).add("A")
msg2 = msg1.copy().add("B")
print(msg1.text, msg1.log)
print(msg2.text, msg2.log)
```

6. מה תדפיס התוכנית הבאה?

```
def closesolc(f, res):
    more = {f(item) for item in res}
    if not more.issubset(res): #checks if res is a subset of more
        res.update(more) #adds items from more into res
        return closesolc(f, res)
    return res

def two_shift(x):
    return x[-2:] + x[:-2]

print(closesolc(two_shift, {"aab", "ccdd"}))
```

7. מה תדפיס התוכנית הבאה?

```
def f3():
    try: raise IndexError
    except ZeroDivisionError: print("Hey")
    finally: print("Ho")

def call_f3_twice():
    try: f3()      #once!
    except: print("Here")
    finally: print("There")
    f3()          #twice!

def to_do_something():
    try: call_f3_twice()
    except Exception:
        print ("Now")
    finally:
        print("Then")

try: to_do_something()
except: print("When you can't")
```

8. מה תדפיס התוכנית הבאה?

```
class Node:
    def __init__(self, data=None, next=None):
        self.data = data
        self.next = next

def du(head,i):
    if i>0:
        return du(head.next,i-1)
    if head:
        return Node(head.data,du(head.next,i))

def pr(head):
    if head:
        print(head.data, end = "->")
        pr(head.next)

x = Node("a", Node("b", Node("c", Node("d",Node("e")))))
pr(du(x,2))
```

9. מה תדפיס התוכנית הבאה?

```
import functools

lst = [1,7,17,9,2,-5]
while(lst):
    f = lambda a,x: a if a < x else x
    val = functools.reduce(f , lst)
    print(val, end = " ")
    lst = list(filter(lambda x: x>val, lst))
```

10. מה תדפיס התוכנית הבאה?

```
x = iter(range(3))
y = iter(range(3))

w = ([next(y,"a"), next(y,"b")] for z in x)
try:
    print(next(w))
    print(next(w))
    print(next(w))
    print(next(w))
    print(next(w))
except:
    print("7")
```

**חלק ב' – שאלות תכנות** (60% -- יש לענות רק על 4 מתוך 6 השאלות, כל שאלה 15 נק')

יש לענות על כל שאלה רק בתוך הקופסה שניתנה בדף. מומלץ להשתמש במחברת הטיוטא כדי לגבש את הפתרון ורק כשהוא מושלם להעתיקו לטופס המבחן. אין צורך בהערות או בתיעוד. יש לכתוב קוד בהיר ויעיל.

1. ריבוע קסם הוא טבלה של מספרים בגודל  $n \times n$  המכילה את כל השלמים בטווח מ-1 עד  $n^2$ . בנוסף מתקיים שסכומם של כל המספרים בכל שורה, בכל עמודה, ובשני האלכסונים של הטבלה הוא זהה. כתבו פונקציה `is_magic_square(mat)` המקבלת רשימה דו-מימדית `mat` בגודל  $n \times n$ , ומחזירה `True` אם `mat` היא ריבוע קסם ו-`False` אחרת.

ניתן להניח שגודל הרשימה הוא אכן ריבוע בגודל  $n \times n$  עבור  $n$  כלשהוא, ושהיא ממילה בדיוק פעם אחת כל מספר בין  $1$  ל- $n^2$  (כנדרש). נותר רק לבדוק שסכום השורות העמודות והאלכסונים זהה. שימו לב: אין צורך לבדוק את סכומי של אחד מהאלכסונים שכן ידוע לגבי ריבועי קסם שסכום זה יהיה תקין אם סכום כל השורות, העמודות והאלכסון השני הוא זהה.

[illegible]

לבסוף, תוחזר הרשימה [6, 4, 7] (או רשימה דומה עם אותם ערכים המופיעים בסדר שונה).

3. כתבו פונקציה `ranksearch(lst, k)` שמוצאת את האיבר ה- $k$  הכי קטן ברשימה `lst` (כלומר שיש בדיוק  $k-1$  איברים קטנים ממנו ברשימה). ניתן להניח שהרשימה `lst` אינה ריקה, ושלכל איבריה יש ערכים שונים זה מזה. ניתן גם להניח ש-  $1 \leq k \leq \text{len}(lst)$ . **שימו לב: הפתרון שלכם חייב להיות רקורסיבי** (שימוש בלולאות בתוכנית יכול לסייע כפי שהן מסייעות ב-quicksort למשל). מותר לכם להשתמש בזיכרון נוסף בעת פתרון השאלה לפי הצורך. רמז: לרוב אין צורך למיין את כל הרשימה על מנת למצוא את האיבר ה- $k$  הכי קטן. דוגמא: הרצת `ranksearch([3, 2, 7, 9, 1], 4)` תחזיר 7, שזהו האיבר הרביעי הכי קטן ברשימה.

|                                                                                                                                      |            |             |
|--------------------------------------------------------------------------------------------------------------------------------------|------------|-------------|
| def ranksearch(lst, k):                                                                                                              |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
|                                                                                                                                      |            |             |
| 2 נק') עבור רשימה באורך $n$ , נבחר $k = n/2$ . מהי סיבוכיות זמן הריצה של הפונקציה במקרה הטוב ובמקרה הגרוע ביותר? (כפונקציה של $n$ ). | טוב ביותר: | גרוע ביותר: |



הנחות: המילון מכיל רק מחרוזות. אין מעגלים בשרשראות שבמילון. ניתן להניח כי המילון אינו ריק (אבל יכול להיות מילון בגודל 1). מותר לשנות את המילון המקורי.

[illegible]

```
f = lambda x: x<3
i = iter([1,2,3,4,5,0,1])
lst = list(DropWhile(f,i))
```

[illegible]

6. ממשו מחלקה Shelf המדמה מדף עליו ניתן להניח חפצים בערמות. יש לתמוך בארבעת הפעולות הבאות:

- **יצירה של אובייקט מהמחלקה** (ללא פרמטרים נוספים)
- **השמת דבר מה על המדף:** `place_item_on(self, item, thing)` תניח את `item` על גבי `thing`. ערכו של `thing` צריך להיות **או** המדף עצמו (אז נוצרת ערימה חדשה על המדף לצד ערימות אחרות שקיימות כבר) **או** חפץ שהונח קודם לכן בראש אחת הערימות. אם לא, יש לזרוק `exception` מסוג `ValueError`. ניתן גם להניח ש `item` אינו מונח כבר על המדף לפני הקריאה להניחו. לאחר הקריאה, `item` נמצא בראש הערימה.
- **הסרת חפץ מראש הערימה:** `remove(self, thing)` תסיר את החפץ הנתון מראש הערימה עליה הוא מונח. אם החפץ הנתון אינו בראש אחת הערימות, יש לזרוק `exception` מסוג `ValueError`.
- **הדפסת המדף:** ראו בהערות לקוד מטה דוגמה לפורמט ההדפסה.  
לדוגמא:

```
s = Shelf()
s.place_item_on("a cup", s)
s.place_item_on("a pencil", "a cup")
print(s)                                #prints: a pencil on a cup.
s.place_item_on("a book", s)
print(s)                                #prints: a pencil on a cup. a book.
s.remove("a pencil")
print(s)                                #prints: a cup. a book.
```

```
class Shelf:
```

(המשך בעמ' הבא)

